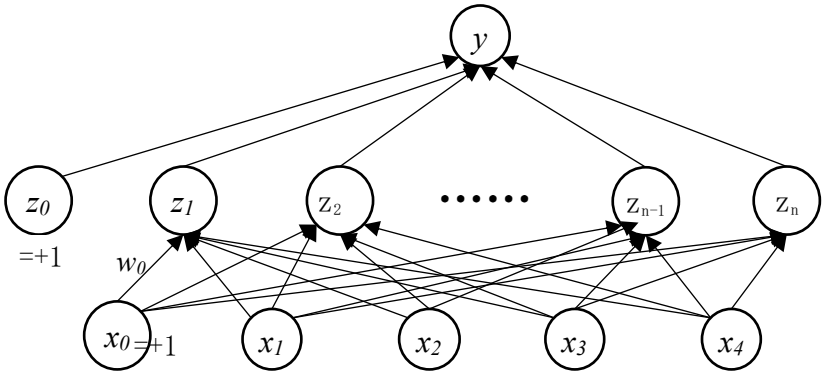
1. Hand-Coding a Neural Network

Design a multilayer perceptron to check if there exists a pair of inputs x_1,x_2,x_3,x_4, where $x_i \in R$, such that their sum is zero. The network should output 1 if any pair $x_i + x_j = 0$ for $i \neq j$, and 0 otherwise. We will use a two-layer neural network like this one:



The output layer will use the **step activation function**, defined as follows:

$$\sigma'(x) = \begin{cases} 1, & if\ x > 0 \\ 0, & otherwise \end{cases}$$

All of the hidden units will use the **impulse activation function**, defined as follows:

$$\sigma(x) = \begin{cases} 1, & if\ x = 0 \\ 0, & otherwise \end{cases}$$

a) (1pt) What can be an appropriate value for n (i.e., the number of hidden units)? Explain the reason.

b) (1pt) Hand-pick a set of weights and biases so that the network correctly implements the desired functionality.

c) (0.5pt) Show that your network correctly classifies the below three sets of inputs:

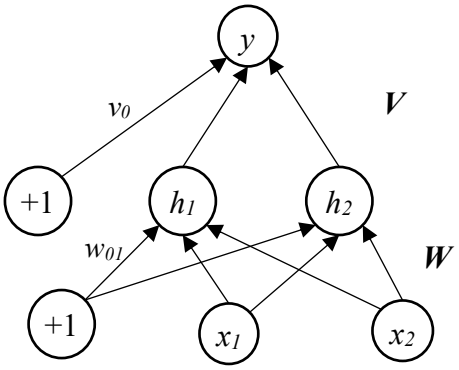
- Input1: $x_1 = 1, x_2 = -1, x_3 = 2, x_4 = 3$
- Input2: $x_1 = 2, x_2 = 3, x_3 = 4, x_4 = 5$
- Input3: $x_1 = 0, x_2 = 5, x_3 = -5, x_4 = 0$

2. Train a neural network for the binary classification problem by hand

We want to train a neural network for the binary classification problem **by hand**. The neural network will have two input features x_1 and x_2, and two hidden units h_1 and h_2. The hidden units h_1 and h_2 will use the ReLU activation, and for final prediction y we will use the sigmoid activation to obtain a prediction between 0 and 1.

Our training example will look like this:

ID	x_1	x_2	t
1	0.2	0.1	0
2	0.8	0.9	1
3	0.5	0.4	0
4	0.7	0.6	1



a) (1pt) Our initial weights and biases will look like this:

$$\mathrm{W} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.4 \end{bmatrix},\ \mathrm{w}_0 = \begin{bmatrix} -0.1 \\ 0.2 \end{bmatrix},\ \mathrm{V} = [1.0 \quad -0.5],\ \mathrm{v}_0 = [0.1]$$

Compute the forward pass for each of the four examples. That is, for each $\mathbf{x}^{(\ell)}$, compute h_1, h_2 and the prediction y. Use the notation a_1 and a_2 to denote the pre-activations values of the hidden states, so that $h_1 = \mathrm{relu}(a_1)$ and $h_2 = \mathrm{relu}(a_2)$. Use z to represent the logit of the prediction, so that $y = \mathrm{sigmoid}(z)$. You may also use the vector representations $\mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}$, $\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$, but always start with the scalar representations first.

b) (1pt) Suppose that we are training the model using the 4th example $(\mathbf{x}^{(4)}, t^{(4)})$, and that we estimate the cross entropy loss function $L(\mathbf{x}^{(4)}, t^{(4)})$. We want to compute the partial derivative of each parameter using backpropagation. Let $\delta_s = \frac{\partial L}{\partial s}$ denote the error signal upon any variable s, please complete the following formulas: (Hint: you can reuse the error signal symbol of the previous one when you complete each formula)

$\delta y =$ $y^{(4)} - t^{(4)}$

$\delta z =$ $\delta y \cdot$?

$\delta v_k =$

$\delta v_0 =$

$\delta h_k =$

$\delta a_k =$

$\delta w_{jk} =$

$\delta w_{0k} =$

c) (0.5pt) Assuming a learning rate of 0.5, perform one iteration of weight update based on the values you obtained from part b)